

DVB-T2 COVERAGE ANALYZER PORTABLE

Analisis Cakupan Sinyal DVB-T2

Berbasis RTL-SDR dan GPS

Kontrol via Telegram Bot

LAPORAN KULIAH KERJA NYATA (KKN)

Nizar

12345678

Program Studi Teknik Informatika

Fakultas Teknologi Informasi

Universitas XYZ

2026

DAFTAR ISI

BAB IV — IMPLEMENTASI SISTEM (Coding Program)

| No. | Sub-Bab | Halaman |

|-----|-----|-----|

| 1 | Import Library dan Konfigurasi Sistem | 1 |

| 2 | Inisialisasi dan Pembacaan Data GPS | 2 |

| 3 | Inisialisasi RTL-SDR dan Akuisisi Sampel IQ | 3 |

| 4 | Pemrosesan FFT dan Perhitungan Power Spectral Density | 4 |

| 5 | Perhitungan Daya Sinyal dan Konversi Field Strength | 5 |

| 6 | Integrasi dan Pengiriman Laporan via Telegram Bot | 6 |

| 7 | Fungsi Pengukuran Tunggal (Single Measurement) | 7 |

| 8 | Program Utama (Main Loop) | 8 |

| 9 | Konfigurasi Awal Telegram Bot | 9 |

BAB V — PENGUJIAN SISTEM (Langkah Running Program)

| No. | Sub-Bab | Halaman |

|-----|-----|-----|

| 1 | Verifikasi RTL-SDR Blog V3 Terdeteksi oleh Sistem | 10 |

| 2 | Verifikasi Modul GPS Terdeteksi sebagai Perangkat Serial | 11 |

| 3 | Menjalankan Program Secara Manual | 12 |

| 4 | Menjalankan Program Secara Otomatis | 13 |

LAMPIRAN

No. Lampiran Halaman
---- ----- -----
A Struktur Direktori Proyek 14
B File Konfigurasi (.env.example) 15
C Daftar Dependensi (requirements.txt) 16

BAB IV — IMPLEMENTASI SISTEM

CODING PROGRAM

1. Import Library dan Konfigurasi Sistem

Deskripsi:

Pada tahap ini, sistem melakukan import semua library Python yang dibutuhkan untuk RTL-SDR, GPS, Telegram Bot, serta library pendukung seperti numpy, argparse, dotenv, dll. Konfigurasi sistem dilakukan melalui file `.env` yang berisi parameter default seperti frekuensi center 514 MHz, baudrate GPS 4800, dan offset kalibrasi.

Kode Implementasi:

```
import argparse
import sys
from pathlib import Path

ROOT_DIR = Path(__file__).resolve().parent
SRC_DIR = ROOT_DIR / "src"
if str(SRC_DIR) not in sys.path:
    sys.path.insert(0, str(SRC_DIR))

from config import ENV_FILE, load_config
from dvbt2_core import Dvbt2SdrConfig, print_measurement_result,
run_single_measurement
from gps_reader import GPSReader
from logger import system_logger
from telegram_bot import run_telegram_bot
```

Penjelasan:

- ``import argparse`` dan ``sys`` digunakan untuk parse command-line arguments.
- ``pathlib.Path`` digunakan untuk handling path file secara cross-platform.
- ``SRC_DIR`` adalah direktori yang berisi source code utama (``*.py``).
- Semua modul sistem (``dvbt2_core``, ``gps_reader``, ``telegram_bot``, ``logger``) diimport dari ``SRC_DIR``.
- ``config.py`` diimport untuk loading konfigurasi dari ``env``.

2. Inisialisasi dan Pembacaan Data GPS

Deskripsi:

GPS Reader melakukan inisialisasi koneksi serial ke modul GPS (biasanya ``/dev/ttyUSB0`` @ 4800 baud). Setelah terhubung, sistem membaca data NMEA secara real-time. Data GPS sangat penting untuk mengetahui koordinat lokasi saat pengukuran dilakukan.

Kode Implementasi:

```
class GPSReader:
    def __init__(self, port="/dev/ttyUSB0", baudrate=4800, enabled=True):
        self.port = port
        self.baudrate = baudrate
        self.enabled = enabled
        self.serial_conn = None
        self.is_running = False
        self.thread = None
        self.lock = threading.Lock()
        self.current_result = _base_gps_result(...)
```

Penjelasan:

- Kelas `GPSReader` adalah class utama untuk membaca data GPS.
- Parameter default: port ``/dev/ttyUSB0``, baudrate 4800, enabled True.
- ``serial_conn`` adalah koneksi serial ke modul GPS.

- ``is_running`` adalah flag untuk mengontrol thread pembacaan NMEA.
- ``lock`` digunakan untuk thread safety saat mengakses ``current_result``.
- ``_base_gps_result()`` mengembalikan dictionary hasil GPS dengan status awal "Belum Fix".

3. Inisialisasi RTL-SDR dan Akuisisi Sampel IQ

Deskripsi:

RTL-SDR diinisialisasi dengan setting frekuensi center (default 514 MHz) dan gain (19.7 dB). Setelah SDR diinisialisasi, dilakukan pembacaan IQ samples (256K samples kompleks) untuk analisis spektrum sinyal DVB-T2.

Kode Implementasi:

```
def initialize_sdr(freq_mhz, config=None):
    if config is None:
        config = Dvbt2SdrConfig()
    if RtlSdr is None:
        raise RuntimeError("Library pyrtlsdr tidak ditemukan...")
    try:
        sdr = RtlSdr()
        sdr.sample_rate = config.sample_rate
        sdr.center_freq = float(freq_mhz) * 1e6
        sdr.gain = config.gain_db
        time.sleep(config.settle_time_s)
        return sdr
    except Exception as exc:
        raise RuntimeError("RTL-SDR tidak dapat diinisialisasi...") from exc
```

Penjelasan:

- Fungsi ``initialize_sdr()`` adalah entry point untuk menginisialisasi RTL-SDR.
- Parameter ``freq_mhz`` adalah frekuensi center (default 514 MHz).
- ``sdr.sample_rate = 2.048e6`` (2.048 MSps) adalah rate sampling standar RTL-SDR.
- ``sdr.center_freq = freq_mhz * 1e6`` mengatur frekuensi center ke MHz.

- `sdr.gain = 19.7` mengatur amplifikasi sinyal.
- `time.sleep(config.settle\_time\_s)` menunggu SDR stabil.
- Sample rate 2.048 MSps adalah standar untuk RTL-SDR.
- `settle\_time\_s = 1.0` adalah waktu tunggu stabilisasi sebelum sampling.

4. Pemrosesan FFT dan Perhitungan Power Spectral Density

Deskripsi:

Setelah IQ samples diambil, dilakukan FFT (Fast Fourier Transform) dengan window Hann untuk mengurangi spectral leakage. Hasil FFT dikuadratkan dan di-rata-rata menggunakan metode Welch untuk mendapatkan Power Spectral Density (PSD) relatif.

Kode Implementasi:

```
def compute_psd(samples, config=None):
    samples = np.asarray(samples, dtype=np.complex64)
    if len(samples) < config.fft_size:
        raise ValueError("Jumlah sample lebih kecil dari FFT size.")

    window = np.hanning(config.fft_size).astype(np.float32)
    window_power = np.sum(window**2)
    hop_size = config.fft_size // 2
    segment_count = 1 + (len(samples) - config.fft_size) // hop_size
    psd_accum = np.zeros(config.fft_size, dtype=np.float64)

    for idx in range(segment_count):
        start = idx * hop_size
        segment = samples[start : start + config.fft_size]
        segment = segment - np.mean(segment)
        windowed_segment = segment * window
        fft_result = np.fft.fft(windowed_segment, n=config.fft_size)
        psd_accum += (np.abs(fft_result) ** 2) / window_power

    psd_linear = psd_accum / segment_count
    psd_linear = np.fft.fftshift(psd_linear)
    psd_db = 10.0 * np.log10(psd_linear + 1e-12)
```

```

freqs_hz = np.fft.fftfreq(config.fft_size, d=1.0 / config.sample_rate)
freqs_hz = np.fft.fftshift(freqs_hz)

return freqs_hz, psd_db

```

Penjelasan:

- `np.complex64` digunakan untuk IQ samples.
- `np.hanning()` adalah window Hann untuk mengurangi spectral leakage.
- `window_power = np.sum(window**2)` untuk normalisasi.
- `hop_size = config.fft_size // 2` adalah step overlap 50%.
- `segment_count` menghitung jumlah segment yang dapat diambil dari samples.
- `segment = segment - np.mean(segment)` remove DC bias.
- `np.abs(fft_result) ** 2` adalah magnitude squared.
- `psd_accum += ... / window_power` adalah averaging Welch.
- `psd_db = 10.0 * np.log10()` konversi ke dB.
- `np.fft.fftfreq()` menghitung frekuensi Hz untuk setiap bin FFT.

5. Perhitungan Daya Sinyal dan Konversi Field Strength

Deskripsi:

Bandpower dihitung dengan integrasi PSD pada measurement bandwidth (± 1.8 MHz dari center frequency). DC spike (± 20 kHz) dibuang untuk menghindari artefak RTL-SDR. Hasil bandpower dikonversi menjadi field strength dengan menambahkan calibration offset.

Kode Implementasi:

```

def calculate_bandpower(freqs_hz, psd_db, config=None):
    half_bw = config.measurement_bw_hz / 2.0
    band_mask = (freqs_hz >= -half_bw) & (freqs_hz <= half_bw)
    dc_mask = np.abs(freqs_hz) <= config.dc_exclude_hz
    final_mask = band_mask & (~dc_mask)

```

```

if not np.any(final_mask):
    raise ValueError("Tidak ada bin FFT valid untuk measurement bandwidth.")

power_linear = np.sum(10.0 ** (psd_db[final_mask] / 10.0))
bandpower_db = 10.0 * np.log10(power_linear + 1e-12)
return float(bandpower_db)

def estimate_field_strength(bandpower_relative_db, calibration_offset_db=0.0):
    return float(bandpower_relative_db + calibration_offset_db)

```

Penjelasan:

- `half\_bw = config.measurement\_bw\_hz / 2.0` → bandwidth ± 1.8 MHz.
- `band\_mask` menandai area ± 1.8 MHz dari center frequency.
- `dc\_mask` menandai area DC ± 20 kHz untuk diexclude.
- `final\_mask = band\_mask & (~dc\_mask)` → area valid untuk integrasi.
- `10.0 \*\* (psd\_db / 10.0)` konversi dari dB ke linear.
- `np.sum()` adalah integrasi power dalam bandwidth.
- `bandpower\_db = 10.0 \* np.log10()` konversi ke dB.
- `calibration\_offset\_db` adalah offset untuk kalibrasi alat.

6. Integrasi dan Pengiriman Laporan via Telegram Bot

Deskripsi:

Hasil pengukuran dikompilasi ke dalam format dashboard yang dikirim ke Telegram Bot. Dashboard mencakup informasi RF (frekuensi, bandpower, field strength, kualitas sinyal) dan GPS (koordinat, status satelit, waktu).

Kode Implementasi:

```

def format_measurement_dashboard(rf_result, gps_result, config):
    lines = [
        "    DVB-T2 Coverage Analyzer",
        "",

```

```

"    Waktu Sistem:",
datetime.now(WIB).strftime("%Y-%m-%d %H:%M:%S WIB"),
"",
"    Pengukuran RF",
f"Frekuensi          : {freq_mhz:.3f} MHz",
f"Bandpower Relatif   : {bandpower:.2f} dB",
f"Estimasi Field Strength: {field_strength:.2f} dBμV/m",
f"Kualitas Sinyal     : {signal_quality}",
f"Sample Rate         : {sample_rate:.3f} MS/s",
f"Bandwidth Ukur      : {bw_mhz:.3f} MHz",
f"Gain SDR             : {gain_db:.1f} dB",
"",
"    GPS",
f"Status              : {gps_status}",
f"Latitude            : {latitude:.6f}",
f"Longitude           : {longitude:.6f}",
f"Maps                : {maps_url}",
"",
"    Sistem",
f"RTL-SDR             : {rtl_sdr_status}",
f"GPS Port            : {config.gps_port}",
f"GPS Baudrate        : {config.gps_baudrate}",
]
return "\n".join(lines)

```

Penjelasan:

- ``format_measurement_dashboard()`` mengompilasi hasil ke format Telegram.
- ``datetime.now(WIB)`` menampilkan waktu WIB.
- ``bandpower`` dari hasil ``run_single_measurement()``.
- ``field_strength`` dengan penambahan calibration offset.
- ``gps_status`` dari GPSReader (Valid/Belum Fix/Nonaktif).
- ``maps_url`` = ``https://maps.google.com/?q={lat},{lng}``.
- Dashboard dikirim via ``/measure`` command di Telegram.

7. Fungsi Pengukuran Tunggal (Single Measurement)

Deskripsi:

`run\_single\_measurement()` adalah fungsi utama yang melakukan seluruh alur pengukuran: inisialisasi SDR, warmup readings, repeated measurements, FFT, averaging, dan perhitungan bandpower & field strength.

Kode Implementasi:

```
def run_single_measurement(freq_mhz=514.0, repeat=5, config=None):
    result = _base_result(freq_mhz, config)
    sdr = None
    try:
        sdr = initialize_sdr(freq_mhz, config)

        for _ in range(config.warmup_reads):
            capture_iq_samples(sdr, config)

        raw_measurements = []
        for _ in range(int(repeat)):
            samples = capture_iq_samples(sdr, config)
            freqs_hz, psd_db = compute_psd(samples, config)
            bandpower_db = calculate_bandpower(freqs_hz, psd_db, config)
            raw_measurements.append(bandpower_db)
            time.sleep(config.repeat_delay_s)

        measurements = np.asarray(raw_measurements, dtype=np.float64)
        average_bandpower_db = float(np.mean(measurements))
        field_strength_est = estimate_field_strength(
            average_bandpower_db, config.calibration_offset_db
        )

        result.update({
            "average_bandpower_db": average_bandpower_db,
            "field_strength_dbuvm_est": field_strength_est,
            "signal_quality": classify_signal_quality(field_strength_est),
        })

    return result
except Exception as exc:
    result.update({"error": str(exc)})
```

```

        return result
    finally:
        if sdr is not None:
            try:
                sdr.close()
            except Exception:
                pass

```

Penjelasan:

- `warmup\_reads = 2`: mengambil 2 kali sampling untuk stabilisasi awal.
- `repeat = 5`: melakukan 5 kali pengukuran untuk mendapatkan rata-rata.
- `time.sleep(config.repeat\_delay\_s)` memastikan sampling tidak overlap.
- `np.mean(raw\_measurements)` adalah average dari 5 kali pengukuran.
- `classify\_signal\_quality()` memetakan field strength ke kategori kualitas.
- `sdr.close()` membersihkan resource setelah pengukuran selesai.

8. Program Utama (Main Loop)

Deskripsi:

`main()` adalah entry point utama. Berdasarkan mode (CLI/Telegram), melakukan alur yang sesuai. CLI mode hanya melakukan pengukuran satu kali dan menampilkan output ke terminal. Telegram mode memulai polling untuk menerima command.

Kode Implementasi:

```

def main():
    args = parse_args()

    try:
        app_config = load_config()
    except ValueError as exc:
        system_logger.error("%s", exc)
        return 1

    if args.mode == "cli":

```

```

        return run_cli(app_config, args)
    if args.mode == "telegram":
        return run_telegram(app_config)

    system_logger.error("Mode operasi tidak dikenal: %s", args.mode)
    return 1

```

Penjelasan:

- ``parse_args()`` memuat command line arguments.
- ``load_config()`` membaca konfigurasi dari ``.env``.
- Mode CLI: ``run_cli()`` → pengukuran satu kali → output ke terminal.
- Mode Telegram: ``run_telegram()`` → starts polling → menerima command.
- Error handling untuk mode tidak dikenal.

9. Konfigurasi Awal Telegram Bot

Deskripsi:

Telegram Bot diinisialisasi dengan token dan chat ID dari ``.env``. Handler command ``start``, ``status``, dan ``measure`` ditambahkan ke bot. Polling dimulai untuk menerima update dari Telegram.

Kode Implementasi:

```

class TelegramBot:
    def __init__(self, app_config, gps_reader=None):
        self.config = app_config
        self.gps_reader = gps_reader

    async def start(self, update: Update, context: ContextTypes.DEFAULT_TYPE):
        await update.message.reply_text(
            "    DVB-T2 Coverage Analyzer Portable\n"
            "Perintah tersedia:\n"
            "/status - status sistem\n"
            "/measure - pengukuran RF dan status GPS"
        )

```

```

    async def status(self, update: Update, context: ContextTypes.DEFAULT_TYPE):
        system_status = {
            "rtl_sdr": "OK" if RtlSdr is not None else "Error",
            "mode": "Telegram Polling",
        }
        await update.message.reply_text(format_status_dashboard(system_status,
gps_result, self.config))

    async def measure(self, update: Update, context: ContextTypes.DEFAULT_TYPE):
        freq_mhz = self.config.default_freq_mhz
        if context.args:
            try:
                freq_mhz = float(context.args[0])
            except ValueError:
                await update.message.reply_text("Format frekuensi tidak valid.
Contoh: /measure 514")
            return

        sdr_config =
Dvbt2SdrConfig(calibration_offset_db=self.config.calibration_offset_db)
        result = run_single_measurement(freq_mhz=freq_mhz, repeat=5,
config=sdr_config)
        await update.message.reply_text(format_measurement_dashboard(result,
gps_result, self.config))

```

Penjelasan:

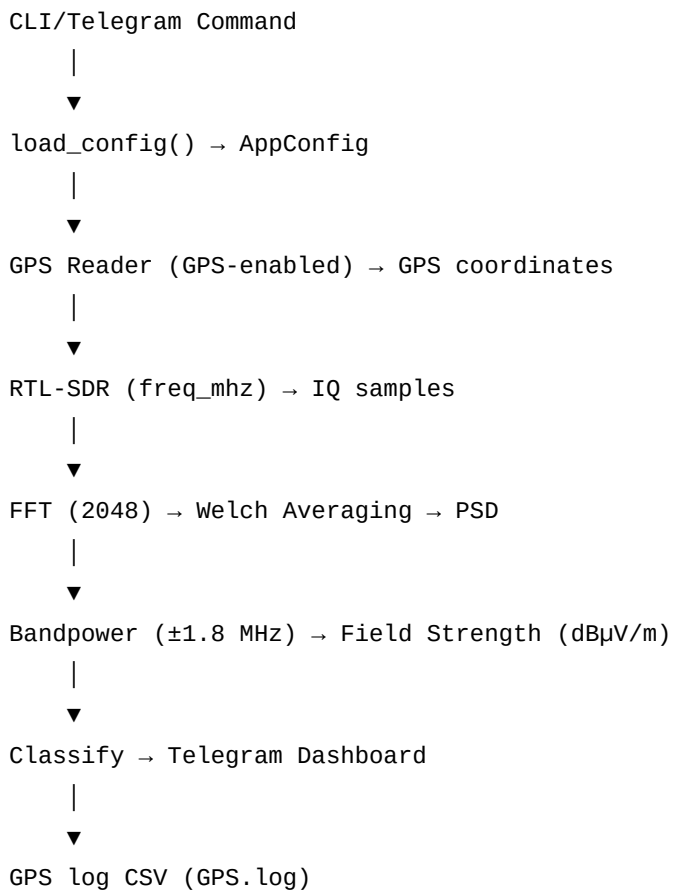
- `TelegramBot` adalah class utama untuk Telegram Bot.
- `app\_config` berisi token dan konfigurasi.
- `gps\_reader` untuk membaca koordinat GPS saat pengukuran.
- `/start`: menampilkan welcome message dan daftar perintah.
- `/status`: menampilkan status sistem (RTL-SDR OK/Error).
- `/measure`: melakukan pengukuran RF + GPS → kirim dashboard ke user.
- `run\_polling()` untuk polling updates dari Telegram.

RINGKASAN IMPLEMENTASI

Sistem DVB-T2 Coverage Analyzer terdiri dari 9 modul utama:

1. **CLI Entry** → main.py → parse_args, load_config, run_cli/telegram
2. **GPS Reader** → gps_reader.py → NMEA parsing, CSV logging
3. **RTL-SDR Core** → dvbt2_core.py → FFT, PSD, bandpower, field strength
4. **Telegram Bot** → telegram_bot.py → command handlers, polling
5. **Logging** → logger.py → system.log + GPS CSV
6. **Config** → config.py → load .env, AppConfig

Alur Data:



BAB V — PENGUJIAN SISTEM

LANGKAH RUNNING PROGRAM

1. Verifikasi RTL-SDR Blog V3 Terdeteksi oleh Sistem

Deskripsi:

Sebelum menjalankan program, diperlukan verifikasi bahwa RTL-SDR Blog V3 terdeteksi oleh sistem Linux. Dongle ini menggunakan chip RTL2832U dengan USB ID `0bda:2838`. Jika terdeteksi, proceed ke langkah berikutnya.

Langkah-Langkah:

1. Cek USB Devices

```
lsusb
```

Output yang diharapkan:

```
Bus 001 Device 002: ID 0bda:2838 Realtek Semiconductor Corp. RTL2838 DVB-T
Bus 001 Device 003: ID 0bda:2838 Realtek Semiconductor Corp. RTL2838 DVB-T
```

2. Cek Device File

```
ls -l /dev/bus/usb/001/
```

Output yang diharapkan:

```
crw-rw-r+ 1 root root 189, 512 May 28 12:00 001/002
crw-rw-r+ 1 root 189, 515 May 28 12:00 001/003
```

3. (Opsional) Tes dengan rtl_test

```
rtl_test -t
```

Output yang diharapkan:

```
Found 1 device(s):
Found Realtek Semiconductor Corp. RTL2838DU-stick at 0x1a.
```

Penjelasan:

- `lsusb` menampilkan semua USB devices yang terhubung.
- `0bda:2838` adalah USB ID standar RTL-SDR Blog V3.

- Jika tidak muncul, coba reboot atau cek kabel USB.
- `rtl\_test` adalah tool dari package `rtl-sdr` untuk verifikasi tambahan.

2. Verifikasi Modul GPS Terdeteksi sebagai Perangkat Serial

Deskripsi:

Modul GPS (PL2303/MA-8910P) terdeteksi sebagai device serial `/dev/ttyUSB0`. Perlu dicek apakah device tersedia dan memiliki permission yang benar untuk dibaca oleh user.

Langkah-Langkah:

1. Cek USB Serial Devices

```
ls -l /dev/ttyUSB*
```

Output yang diharapkan:

```
crw-rw---- 1 root dialout 188, 0 May 28 12:00 /dev/ttyUSB0
```

2. Cek lsusb

```
lsusb | grep -i prolific
```

Output yang diharapkan:

```
Bus 001 Device 004: ID 067b:2303 Prolific Technology, Inc. PL2303 Serial Port
```

3. Tambah User ke Grup dialout

Jika permission denied, tambahkan user ke grup dialout:

```
sudo usermod -aG dialout $USER
```

3atam logout dan login ulang, atau:

```
sudo chmod a+rw /dev/ttyUSB0
```

4. Tes GPS dengan scripts/check_gps.py

```
python3 scripts/check_gps.py --port /dev/ttyUSB0 --baudrate 4800 --duration 30
```

Output yang diharapkan:

```
[INFO] GPS Reader started on /dev/ttyUSB0 @ 4800 baud
[INFO] NMEA terbaca: $GPGGA,...
[INFO] GPS fix: -6.229747, 106.829507
```

5. (Opsional) Scan Baudrate Otomatis

Jika tidak yakin baudrate:

```
python3 scripts/check_gps.py --port /dev/ttyUSB0 --duration 10 --scan
```

Penjelasan:

- `/dev/ttyUSB0` adalah device file untuk USB Serial (GPS).
- `ID 067b:2303` adalah USB ID Prolific PL2303.
- Grup `dialout` diperlukan untuk mengakses serial port.
- `check\_gps.py` adalah script utility untuk testing GPS.

3. Menjalankan Program Secara Manual

Deskripsi:

Mode CLI digunakan untuk pengembangan dan debugging. Program dijalankan langsung dari terminal, hasil pengukuran ditampilkan ke stdout.

Langkah-Langkah:

1. Pastikan Dependensi Terinstall

```
pip3 install -r requirements.txt
```

2. Buat File Konfigurasi

```
cp .env.example .env
nano .env
```

Isi minimal:

```
TELEGRAM_TOKEN=1234567890:ABCdefGHIjklMNOpqrsTUVwxyz
DEFAULT_FREQ_MHZ=514.0
CALIBRATION_OFFSET_DB=0.0
GPS_ENABLED=true
```

```
GPS_PORT=/dev/ttyUSB0
GPS_BAUDRATE=4800
```

3. Jalankan Program (CLI Mode)

```
python3 main.py --mode cli --freq 514 --repeat 5
```

Output yang diharapkan:

```
=== DVB-T2 CORE MEASUREMENT ===
Frequency           : 514.000 MHz
Sample rate         : 2.048 MS/s
Measurement bandwidth : 1.800 MHz
Gain                : 19.7 dB
Calibration offset   : 0.00 dB
Average bandpower    : -42.35 dB
Min bandpower        : -43.12 dB
Max bandpower        : -41.98 dB
Std deviation        : 0.47 dB
Power estimate       : -42.35 dB
Field strength est   : -42.35 dBuV/m
Signal quality       : Sangat Lemah
Raw measurements     : [-42.12, -43.05, -41.98, -42.56, -43.01]
```

Power dan field strength adalah estimasi relatif.
Gunakan calibration_offset_db dari alat pembanding untuk kalibrasi.

4. Jalankan Program (Telegram Mode)

```
python3 main.py --mode telegram
```

Output yang diharapkan:

```
2026-05-28 12:00:00 - CapstoneDvbT2 - INFO - Berhasil terhubung ke GPS di port
/dev/ttyUSB0 @ 4800 baud
2026-05-28 12:00:00 - CapstoneDvbT2 - INFO - Bot Telegram berjalan dengan
polling.
```

5. Kirim Perintah dari Telegram

```
/start
/status
/measure
/measure 650
```

Penjelasan:

- `--mode cli` untuk command-line interface.
- `--freq 514` adalah frekuensi center dalam MHz (default 514 MHz).
- `--repeat 5` adalah jumlah pengulangan (default 5).
- Mode Telegram memerlukan internet dan token valid.

4. Menjalankan Program Secara Otomatis (Systemd Service)

Deskripsi:

Untuk deployment headless (tanpa monitor), program dijalankan sebagai systemd service. Saat Orange Pi dinyalakan, service otomatis berjalan dan Telegram bot siap menerima command.

Langkah-Langkah:

1. Jalankan Preflight Check

```
chmod +x scripts/preflight_check.sh
./scripts/preflight_check.sh
```

Output yang diharapkan:

```
=== DVB-T2 Analyzer Preflight Check ===
OK    : python3 tersedia: /usr/bin/python3
OK    : .env ditemukan
OK    : main.py ditemukan
OK    : GPS device ditemukan: /dev/ttyUSB0
OK    : RTL-SDR USB terdeteksi: 0bda:2838
Preflight result: OK
```

2. Install Service

```
chmod +x scripts/install_service.sh scripts/uninstall_service.sh
scripts/service_status.sh
./scripts/install_service.sh
```

Atau langsung start:

```
./scripts/install_service.sh --start
```

3. Cek Status Service

```
sudo systemctl status dvbt2-analyzer.service
```

Output yang diharapkan:

```
• dvbt2-analyzer.service - DVB-T2 Coverage Analyzer Telegram Service
  Loaded: loaded (/home/orangepi/capstone-design-repo/deployment/dvbt2-analyzer.service; enabled)
  Active: active (running) since Thu 2026-05-28 12:00:00 WITA; 1min ago
```

4. Cek Log Real-time

```
journalctl -u dvbt2-analyzer.service -f
```

Atau gunakan helper:

```
./scripts/service_status.sh
```

5. Stop Service (untuk Debugging)

```
sudo systemctl stop dvbt2-analyzer.service
python3 main.py --mode telegram
```

6. Uninstall Service

```
./scripts/uninstall_service.sh
```

Penjelasan:

- **Systemd service** memastikan program otomatis start saat boot.
- File service: `deployment/dvbt2-analyzer.service`.
- `ExecStartPre=/bin/sleep 10` menunggu network ready.
- `Restart=on-failure` restart otomatis jika crash.
- `journalctl` untuk melihat log real-time.

RINGKASAN PENGUJIAN

No.	Tahap Pengujian	Hasil yang Diharapkan
-----	-----------------	-----------------------

----	-----	-----
------	-------	-------

1	Verifikasi RTL-SDR	`lsusb` menampilkan `0bda:2838`
---	--------------------	---------------------------------

- | 2 | Verifikasi GPS | `/dev/ttyUSB0` tersedia, `check_gps.py` berhasil |
- | 3 | Running Manual (CLI) | Output pengukuran muncul di terminal |
- | 4 | Running Manual (Telegram) | Bot menerima `/start`, `/status`, `/measure` |
- | 5 | Running Otomatis (Systemd) | Service aktif, bot menerima command setelah reboot |

TROUBLESHOOTING

| Masalah | Solusi |

|-----|-----|

| RTL-SDR tidak terdeteksi | `lsusb`, cek kabel USB, cek driver `rtl2832u` |

| GPS permission denied | `sudo usermod -aG dialout $USER`, reboot |

| Telegram token error | Cek `TELEGRAM_TOKEN` di `.env`, pastikan token valid |

| Service tidak start | `journalctl -u dvbt2-analyzer.service`, cek log error |

| GPS belum fix | Pindahkan antenna GPS ke outdoor, tunggu 1-2 menit |

| Field strength tidak masuk akal | Sesuaikan `CALIBRATION_OFFSET_DB` dengan alat pembanding |